

Lecture 2

2. DATA TYPES AND OPERATORS

2. ТИПЫ ДАННЫХ И ОПЕРАТОРЫ

Roadmap: Data Types and Operators in Java

Same language. Different kinds of data. Powerful operators. Bigger possibilities.

Write.
Compile.
Solve real problems.

Classification of Data Types in Java

Java Data Types

Primitive Types

(basic types, store values directly)
8 types

byte	8 bits (-128 .. 127)
short	16 bits (-32,768 .. 32,767)
int	32 bits ($\approx -2 \cdot 10^9 \dots 2 \cdot 10^9$)
long	64 bits ($\approx -9 \cdot 10^{18} \dots 9 \cdot 10^{18}$)
float	32 bits (floating-point numbers)
double	64 bits (floating-point numbers)
char	16 bits (Unicode character)
boolean	true / false

Reference Types

(store references to objects)
Classes, interfaces, arrays, etc.

String	Text data (e.g. "Hello")
Arrays	Collections of values (e.g. int[])
Classes	Objects of classes (e.g. Student)
Interfaces	Interface types (e.g. List)
Enums	Enumerations (e.g. DayOfWeek)
Records	Data carriers (record) (since Java 16)
Other Objects	Any other objects (e.g. Teacher, Course, Scanner, etc.)

Data types define what kind of values can be stored and what operations can be performed.

Classification of Operators in Java

Java Operators

Arithmetic Operators

(perform mathematical calculations)

+	addition
-	subtraction
*	multiplication
/	division
%	remainder (modulo)
++	increment
--	decrement

```
int sum = a + b;  
double avg = total / n;
```

Assignment Operators

(assign values)

=	assign
+=	add and assign
-=	subtract and assign
*=	multiply and assign
/=	divide and assign
%=	remainder and assign

```
int count = 0;  
count += 1;
```

Comparison Operators

(compare values)

==	equal to
!=	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

```
if (score >= 50) {  
    pass = true;  
}
```

Logical Operators

(combine conditions)

&&	logical AND
	logical OR
!	logical NOT

```
if (passed &&  
    attendance >= 70) {  
    eligible = true;  
}
```

Bitwise Operators

(operate on bits)

&	bitwise AND
	bitwise OR
^	bitwise XOR
~	bitwise NOT
<<	left shift
>>	right shift
>>>	unsigned right shift

```
int mask = 1 << 3;
```

Ternary Operator

(conditional expression)

```
?: condition ? valueIfTrue :  
    valueIfFalse
```

```
String result =  
    (score >= 50) ? "Pass" :  
    "Fail";
```

Type Operators

(type-related operations)

instanceof check type
(type) cast (explicit conversion)

```
if (obj instanceof  
    Student) {  
    Student s = (Student) obj;  
}
```

Other Operators

(miscellaneous)

new create an object
. access field/method
[] array indexing
:: method reference
-> lambda operator
(introduced in Java 8)

```
Student s = new Student();  
System.out::println;
```

Operators allow us to process data, make decisions, and control the flow of our programs.

We use arithmetic operators to solve math problems.

We use assignment operators to give values to variables.

We use comparison operators to compare values.

We use logical operators to combine conditions.
We use bitwise operators to work with bits.

We use the ternary operator to choose one of two values.

We use type operators to check or convert data types.

We use other operators to create objects and work with classes and arrays.

Java Versions, Data Types, and Variables

What is the latest version? What types and variables are available? When were they introduced?



1 Java Version

✓ Latest version (as of Sept 2026)

Java SE 27

Released: September 15, 2026

★ Current LTS version

Java SE 25 (LTS)

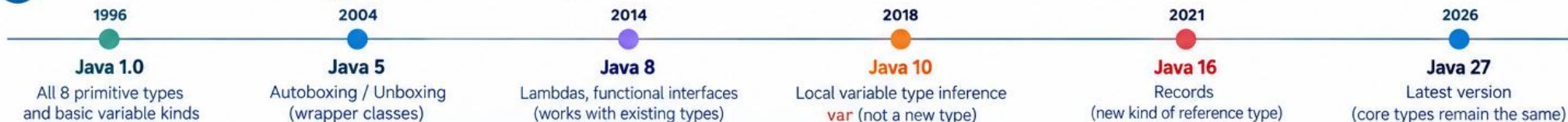
Long-Term Support

Where to find official information?

🔗 Java SE Downloads (Oracle)
<https://www.oracle.com/java/technologies/downloads/>

🔗 Java Language Specification (Java SE 27)
<https://docs.oracle.com/javase/specs/jls/se27/html/index.html>

4 Timeline: When Were Types and Variables Introduced?



Official documentation:

Java Language Specification (JLS) – <https://docs.oracle.com/javase/specs/jls/se27/html/index.html> | Java Tutorials – <https://docs.oracle.com/javase/tutorial/>

Same foundation.
More possibilities.

2 Data Types in Java

Java has two categories of data types:

Primitive Types

(8 basic types, introduced in Java 1.0)

Type	Size	Range (approximate)
byte	8 bit	-128 .. 127
short	16 bit	-32,768 .. 32,767
int	32 bit	$\approx \pm 2 \cdot 10^9$
long	64 bit	$\approx \pm 9 \cdot 10^{18}$
float	32 bit	(floating-point)
double	64 bit	(floating-point)
char	16 bit	Unicode character
boolean	n/a	true / false

```
// Examples
int age = 19;
double grade = 87.5;
char group = 'A';
boolean active = true;
```

Reference Types

(store references to objects)

Class types	Objects of classes (e.g. String, Student)
Interface types	Objects of interfaces (e.g. List)
Array types	Arrays (e.g. int[], String[])
Enum types	Enumerations (e.g. DayOfWeek)
Record types	Records (since Java 16) (e.g. record Student(...))
Other reference types	Any other objects (e.g. Teacher, Course)

```
// Examples
String name = "Anna";
Student student = new Student();
int[] grades = {85, 90, 78};
```

3 Kinds of Variables in Java

Variables can be classified by where they are declared and how they are used.

1. Instance Variables (Fields)

- Declared in a class, outside any method
- Belong to an object (instance)
- Have a default value

```
class Student {
    String name;
    int age;
}
```

2. Class Variables (Static Fields)

- Declared with static
- Belong to the class (shared by all objects)
- Have a default value

```
class Student {
    static int count;
}
```

3. Local Variables

- Declared inside a method, constructor or a block
- Exist only within that block
- Do not have a default value (must be initialized)

```
void calculate() {
    int sum = 0;
    double avg = 0.0;
}
```

4. Method Parameters

- Variables that receive values when a method is called
- Treated as local variables

```
void setGrade(
    double grade) {
    ...
}
```

Версия Java 1.0 появилась в 1996 г. и включала все 8 типов примитивных данных

Версия Java 5.0 появилась в 2004 г. и включала обертки классов

Java is a statically typed programming language

It means that every variable has a data type, and we define the data type when we declare a variable.



- 1 Java is a statically typed programming language.
- 2 It means that **every variable** has a data type.
- 3 We define the data type when we declare a variable.
- 4 For example, we use `int` for integer numbers.
- 5 We use `String` for **text**.
- 6 Java checks data types during compilation.

Example:

```
int age = 20;           // integer
double gpa = 3.67;      // decimal
boolean active = true;  // true/false
char group = 'A';       // one character
String name = "Anna";   // text
```

Wrong type - compilation error:

```
int age = "twenty";
```

✗ Error: incompatible types
String cannot be converted to int

Other statically typed languages



Java
(1995, Sun Microsystems)



C++
(1985, Bjarne Stroustrup)



C#
(2000, Microsoft)



TypeScript
(2012, Microsoft)



C
(1972, Dennis Ritchie)



Kotlin
(2011, JetBrains)



Swift
(2014, Apple)



Go (Golang)
(2009, Google)

These languages also require a data type for every variable.

In Java, every variable has a data type.

On the right, we can see other statically typed programming languages.

Why is strict (static) typing useful?



Fewer errors

Type errors are found during compilation, not at runtime.



Better code quality

The code is clearer and easier to understand.



Improved performance

The compiler can optimize code for specific types.



Easier maintenance

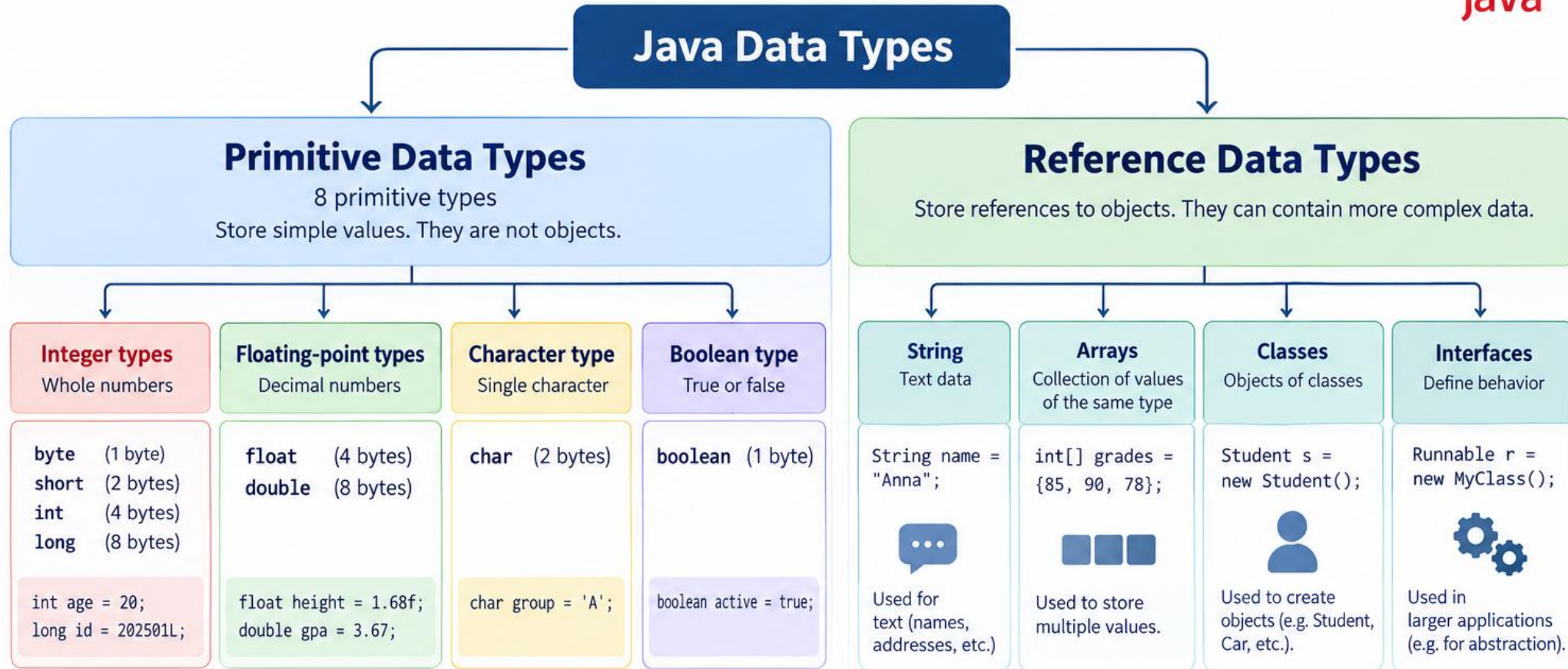
Large projects are more reliable and easier to refactor.

Summary

- Java is a statically typed language.
- Every variable has a data type.
- We define the data type when we declare a variable.
- We use `int` for integers, `String` for text, and other types for other data.
- Java checks data types during compilation.
- Strict typing helps us write safer, clearer, and more reliable programs.

Classification of Data Types in Java

We have two main groups of data types in Java: primitive data types and reference data types.



- Primitive types store the actual value.
- Reference types store a reference (address) to an object in memory.
- String, arrays, classes, and interfaces are all reference types.

Why do we have different data types?

- Different data represents different kinds of information.
- Different types use different amounts of memory.
- Using the correct type helps us write more efficient and reliable programs.

As we said before, Java has eight primitive data types.

Four of them are integer types: byte, short, int, and long.

Two types are used for decimal numbers: float and double.

The char type is used for a single character.

The boolean type is used for true or false values.

Java also has reference data types, such as String, arrays, classes, and interfaces.

Example: Student

A student has different information. We use different data types to store this information.

Different data types help us store different kinds of data efficiently.

Field name	Java type	Description	Example value
id	long	Unique student ID (large number)	2025001234L
surname	String	Student's last name	"Smith"
name	String	Student's first name	"Anna"
age	byte	Student's age (small number)	20
yearOfStudy	short	Current year of study	3
numberOfCourses	int	Number of courses	7
totalLoginCount	long	Number of logins to the system	125000L
height	float	Student's height in meters	1.68f
gpa	double	Grade point average	3.67
groupLetter	char	Group letter	'A'
isActive	boolean	Is the student active?	true
grades	int[]	Array of course grades	{85, 90, 78}

In this example, we can see that each field has a specific data type.

For example, we use String for the student's name, byte for age, and double for GPA.

We choose a data type according to the information we want to store.

Student Table (Sample Data)

ID	ID	Surname	Name	Age	Year	Courses	Logins	Height	GPA	Group	Active	Grades
	2025001234	Smith	Anna	20	3	7	125000	1.68	3.67	A	true	[85, 90, 78]
	2025005678	Johnson	Michael	21	2	6	54321	1.82	3.12	B	true	[76, 88, 91]
	2025009012	Williams	Emily	19	1	5	9876	1.60	3.95	C	false	[92, 95, 89]

The same concept can be used in your own programs. Choose the correct data type for each piece of information.



Key Concepts

1 Data

Facts and information about real-world objects (e.g. students, teachers, courses).

2 Variable

A named memory location used to store a value.

```
int age;  
// variable declaration
```

3 Value

The actual data stored in a variable.

```
age = 20;  
// value 20 is stored
```

4 Data Type

Specifies what kind of values a variable can store and what operations are allowed.

```
int age = 20;  
// type is int
```

For example, we have a student and a teacher.

They have different information: name, age, grade, department, and other data.

Example: Student and Teacher



Student

Name:	Anna
Age:	20
Average grade:	87.5
Enrolled:	true
Group:	"A"



Teacher

Name:	Dr. Smith
Age:	45
Department:	Computer Science
Academic title:	Professor
Active:	true

Java Representation

```
// Student  
String studentName = "Anna";  
int studentAge = 20;  
double averageGrade = 87.5;  
boolean enrolled = true;  
char group = 'A';  
  
// Teacher  
String teacherName = "Dr. Smith";  
int teacherAge = 45;  
String department = "Computer Science";  
String academicTitle = "Professor";  
boolean active = true;
```

We use variables to store this information in Java.

Every variable has a data type.

For example, we use String for a name, int for age, and boolean for true or false.

Then the program can process this data.

From Real World to Program



Real-world data

Information about students, teachers, courses in the university



Java data

Represented using variables with specific data types



Program

We process data using operators, conditions, loops, etc.



Key idea: Real-world information is stored in a Java program using variables with appropriate data types. Types ensure that we can store the right values and perform valid operations.

"Good programs model the real world."

Key Definitions

1 Variable

A variable is a named memory location in a program used to store a value.

2 Value

A value is the actual data stored in a variable (e.g. 19, 87.5, 'A', true).

3 Data type

A data type specifies what kind of values a variable can store and what operations can be performed on those values.

Example: Student Information

```
int age = 19;           // age of the student
double grade = 87.5;    // average grade
char group = 'A';       // group (letter)
boolean enrolled = true; // enrollment status
```

Explanation

Variable name	Data type	Value	Meaning
age	int	19	Student's age (integer number)
grade	double	87.5	Average grade (floating-point number)
group	char	'A'	Group (single character)
enrolled	boolean	true	Enrollment status (true or false)

Type Determines

- What values can be stored in the variable.
- What operations can be performed with these values.
- How much memory is needed.
- The meaning of the data in the program.



The type defines the allowed values and operations.

For example, you can perform arithmetic operations with int and double, but not with boolean. Each type has its own rules.

A variable stores a value.

Every variable has a data type.

The data type defines what values we can store.

For example, age is a variable, int is its data type, and 19 is its value.

We can use different data types for different information.

A variable has a name, a data type, and a value.

4

Primitive and Reference Types

Two main groups of types in Java

Different types.
Different memory.
Different use cases.

1 Primitive Types

Primitive types store **simple values** directly in memory.

- There are 8 primitive types in Java.
- They are not objects.
- They have fixed size and a specific range of values.
- They are stored in the stack (variable contains the value).

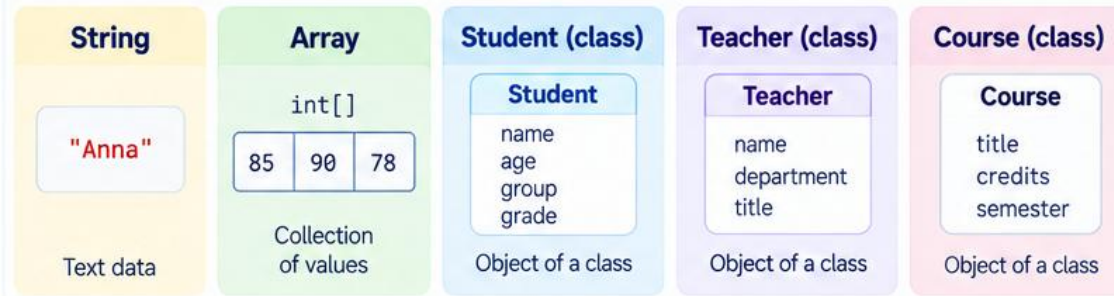
Type	Size	Range (approximate)	Example
byte	8 bit	-128 .. 127	<code>byte b = 100;</code>
short	16 bit	-32,768 .. 32,767	<code>short s = 1000;</code>
int	32 bit	$\approx -2 \cdot 10^9 \dots 2 \cdot 10^9$	<code>int age = 20;</code>
long	64 bit	$\approx -9 \cdot 10^{18} \dots 9 \cdot 10^{18}$	<code>long id = 1_000_000L;</code>
float	32 bit	(floating-point)	<code>float temperature = 23.5f;</code>
double	64 bit	(floating-point)	<code>double grade = 87.5;</code>
char	16 bit	Unicode character	<code>char group = 'A';</code>
boolean	n/a	true / false	<code>boolean active = true;</code>

```
// Example: primitive variables
int age = 20;
double grade = 87.5;
char group = 'A';
boolean enrolled = true;
```

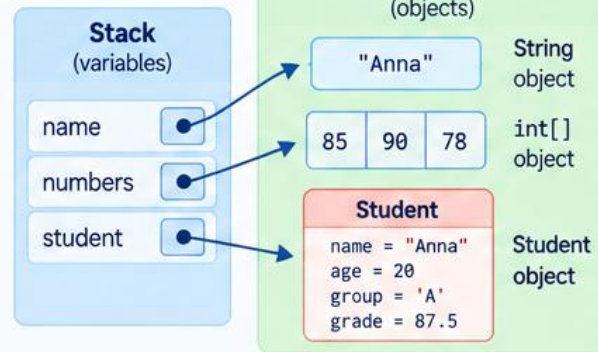
2 Reference Types

Reference types store **references to objects** in memory.

- Include classes, interfaces, arrays, and other objects.
- The variable stores a reference (address), not the object itself.
- Objects are created in the heap memory.
- Examples: String, arrays, Student, Teacher, Course, etc.



Memory model



```
// Example: reference variables
String name = "Anna";
int[] numbers = {85, 90, 78};

Student student = new Student();
student.name = "Anna";
student.age = 20;
student.group = 'A';
student.grade = 87.5;

Teacher teacher = new Teacher();
Course course = new Course();
```

As we can see on this slide – **Primitive types store simple values.**

A reference variable stores a reference to an object.



Key idea: Primitive types store **values** directly. Reference types store references to objects.

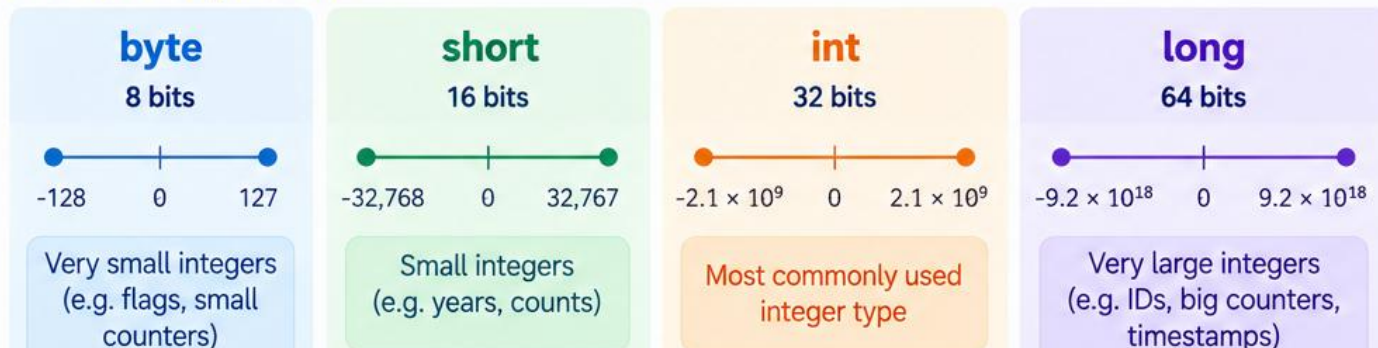
The type of a variable determines what values can be stored and what operations can be performed with those values.

Integer Types in Java

Type	Size (bits)	Range of values (inclusive)	Example
byte	8	-128 .. 127	byte b = 100;
short	16	-32,768 .. 32,767	short s = 20000;
int	32	-2,147,483,648 .. 2,147,483,647	int age = 19; int students = 25;
long	64	-9,223,372,036,854,775,808 .. 9,223,372,036,854,775,807	long studentId = 20260001234L;

Type	Bits	Number of values	Range
byte	8	$2^8 = 256$	-2^7 to $2^7 - 1 = -128$ to 127
short	16	$2^{16} = 65\,536$	-2^{15} to $2^{15} - 1 = -32,768$ to $32,767$
int	32	$2^{32} = 4\,294\,967\,296$	-2^{31} to $2^{31} - 1 = -2,147,483,648$ to $2,147,483,647$
long	64	$2^{64} = 18\,446\,744\,073\,709\,551\,616$	-2^{63} to $2^{63} - 1 = -9,223,372,036,854,775,808$ to $9,223,372,036,854,775,807$

Visual Comparison



```
int age = 20;
```

int is a primitive type.
age stores the value 20.

```
Student student = new Student();
```

Student is a reference type.
student refers to an object.

in the JVM.

Easy to use

As we know, 2 to the power of 8 is 256.

It can store values from -32,768 to 32,767.

So, 8 bits can represent 256 different values.

The int type uses 32 bits.

The byte type uses 8 bits.

It can store values from about -2 billion to 2 billion.

In Java, byte can store numbers from -128 to 127.

The long type uses 64 bits.

The short type uses 16 bits.

We use long for very large integer numbers.



What are floating-point types?

- Used to store real (decimal) numbers.
- Java provides two floating-point types: **float** and **double**.
- They follow the IEEE 754 standard.
- Suitable for measurements, scientific calculations, averages, percentages, etc.

Comparison: float vs double

Property	float	double
Size (bits)	32	64
Approximate range	$\pm 1.4 \times 10^{-45} \dots \pm 3.4 \times 10^{38}$	$\pm 4.9 \times 10^{-324} \dots \pm 1.8 \times 10^{308}$
Precision (significant digits)	~ 6–7 digits	~ 15–16 digits
Default value	0.0f	0.0
Literal example	3.14f	3.14

Example: Student grades and attendance

```
// Student information
double grade = 87.5;      // average grade
double attendance = 92.4; // attendance percentage

// Using float (with 'f' suffix)
float temperature = 23.5f;
float piApprox = 3.14f;
```

Memory and precision (visual comparison)

float
(32 bits)



double
(64 bits)



Java has two types for decimal numbers: float and double.

float uses 32 bits.

double uses 64 bits.

double is more precise than float.

We usually use double for decimal numbers.

For float, we add f after the number.

For example: float temperature = 23.5f;

For double, we do not need f: double grade = 87.5;

Examples from the educational process



Average grade

double grade = 87.5;



Attendance

double attendance = 92.4;



Result (percentage)

double passRate = 95.6;



Key idea: Use **double** for most calculations to get higher precision and avoid rounding errors.
Use **float** when you need to save memory and precision is not critical.



1 char Type

- Stores a single character (one UTF-16 code unit).
- Size: 16 bits (2 bytes).
- Range: '\u0000' (0) to '\uFFFF' (65,535).
- Used to store letters, digits, symbols and Unicode characters.

```
// Example: group of the student
char group = 'A';
char initial = 'N';
char symbol = '#';
char unicodeChar = 'X'; // Unicode character
```

UTF-16 Representation

Character	Unicode code	Decimal value
'A'	\u0041	65
'0'	\u0030	48
'#'	\u0023	35
'X'	\u0058	88
'😊'	\uD83D\uDE0A	55357

i A char variable stores one UTF-16 code unit. Some characters (e.g. 😊) use two code units (a surrogate pair).

2 boolean Type

- Stores a logical value: **true** or **false**.
- Size: typically 1 bit (JVM may use 1 byte internally).
- Used for conditions, flags and logical decisions.
- Default value: **false**.

```
// Example: student status
boolean enrolled = true;
boolean passed = false;
boolean active = true;
```

Boolean Values

Value	Meaning
true	Condition is true, yes, 1
false	Condition is false, no, 0



A boolean variable can store only two values: true or false.

Example: Student Information



```
// Student class (simplified)
class Student {
    String name;
    char group; // e.g. 'A'
    boolean enrolled; // is enrolled
    boolean passed; // exam result
}
```

```
Student s = new Student();
s.name = "Anna";
s.group = 'A';
s.enrolled = true;
s.passed = false;
```

Student Data

Name	Anna
Group	'A'
Enrolled	true
Passed exam	false

char stores one character.

For example, char group = 'A';

We write a char value in single quotes.

char uses 16 bits in Java.

boolean stores a logical value.

It has only two values: true or false.

For example, boolean enrolled = true;

We use boolean for conditions and decisions.

7.1

char Type: Storing a Single Character

Used to store one character (letter, digit, symbol)

We store the student's grade in a char variable.

For example, the grade is A.

The program checks this value.

If the grade is A, the program displays “Excellent”.

If the grade is F, the program displays “Failed”.

Key Information

- **Stores:** one character (one UTF-16 code unit).
- **Size:** 16 bits (2 bytes).
- **Range:** '\u0000' (0) to '\uFFFF' (65,535).
- **Used for:** letters, digits, symbols and any Unicode characters.
- Write char values in single quotes: 'A'.

Examples of char values

'A' letter	'7' digit	'+' symbol
' ' space	'€' Unicode character	'😊' emoji (character)

Memory

char ch = 'A';

Practical Examples

1 Student grade

Store a letter grade and show a message based on the value.

```
char grade = 'A';
if (grade == 'A') {
    System.out.println("Excellent");
} else if (grade == 'B') {
    System.out.println("Good");
} else if (grade == 'C') {
    System.out.println("Satisfactory");
} else {
    System.out.println("Failed");
}
```

Output:
Excellent



2 Menu option

Use a character to select a menu action.

```
char choice = 'S'; // user input
if (choice == 'S') {
    saveData();
    System.out.println("Data saved.");
} else if (choice == 'E') {
    exitProgram();
    System.out.println("Goodbye!");
} else {
    System.out.println("Invalid option.");
}
```

Output:
Data saved.

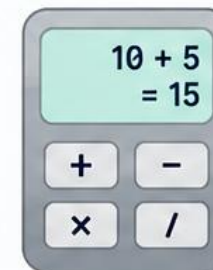


3 Mathematical operator

Use a character to store an operator in a simple calculator.

```
double a = 10;
double b = 5;
char op = '+';
double result = 0;
if (op == '+') {
    result = a + b;
} else if (op == '-') {
    result = a - b;
} else if (op == '*') {
    result = a * b;
} else if (op == '/') {
    result = a / b;
}
System.out.println("Result = " + result);
```

Output:
Result = 15.0



Here we use char to select a menu option.

The user enters one character: S, L, or E.

The program checks the character and performs an action.

For example, S means Save and E means Exit.



Key idea: Use **char** when you need to store and work with a single character. It is useful for codes, grades, menu options, symbols and many real-world applications.

“One character can make a decision.”

8 Literals, Variables and Constants

Writing values, using variables and constants in Java

1 Literal Values in Java

A literal is a fixed value written directly in the code.

Type	Examples of literals	Description
Integer	0, 25, -10, 1_000, 2026L	Whole numbers (see suffix L)
Floating-point	3.14, 0.5, 1_000.0, 2.5F	Decimal numbers (see suffix F)
Character	'A', '7', ' ', '\n'	Single character (UTF-16)
String	"Anna", "CS", "A1"	Sequence of characters
Boolean	true, false	Logical values

2 Suffixes and Underscores

Long integer literal

```
long id = 20260001234L;
```

- Use **L** or **l** for long literals.
- Without a suffix, an integer literal is of type `int` by default.

Underscore in numeric literals (Java 7+)

```
int students = 1_000;  
long population = 20_000_000L;  
double value = 1_234.56;
```

```
final int MAX_STUDENTS = 30;  
final double PI = 3.14159;  
final int DAYS_IN_WEEK = 7;
```

A constant is a value that cannot be changed.

In Java, we use the keyword `final` to create a constant.

For example: `final int MAX_STUDENTS = 30;`

Here, `MAX_STUDENTS` is a constant.

Its value is 30.

We cannot change this value later in the program.

Constants are useful for values that must stay the same.

3 Example: Variables and Literals

```
// Student course information  
int credits = 5;           // integer literal  
double passScore = 50.0;  // floating-point literal  
char group = 'A';         // character literal  
String course = "OOP Java"; // string literal  
boolean active = true;    // boolean literal  
  
long studentId = 20260001234L; // long with suffix L  
float weight = 65.5F;          // float with suffix F  
int maxSeats = 1_000;          // using underscore
```

4 Constants

A constant is a variable whose value cannot be changed. We use the keyword `final`.

```
final int MAX_STUDENTS = 30;  
final double PASS_SCORE = 50.0;  
final String UNIVERSITY = "ENU";
```



The value of a `final` variable cannot be changed after initialization.

5

```
int credits = 5;           // number of credits  
double passScore = 50.0;  // passing score  
final int MAX_STUDENTS = 30; // constant value
```



Key idea

We use constants for values that do not change. In Java, we use the keyword `final`.



Same data. Different types.
Be careful with conversion.

Type conversion means changing a value from one data type to another.

We have two main types of conversion: widening and narrowing.

Widening converts a smaller type to a larger type.

Java does widening automatically.

For example, we can convert int to double automatically.

Narrowing converts a larger type to a smaller type.

For narrowing, we must specify the new type.

This is called casting.

Some data can be lost during casting.

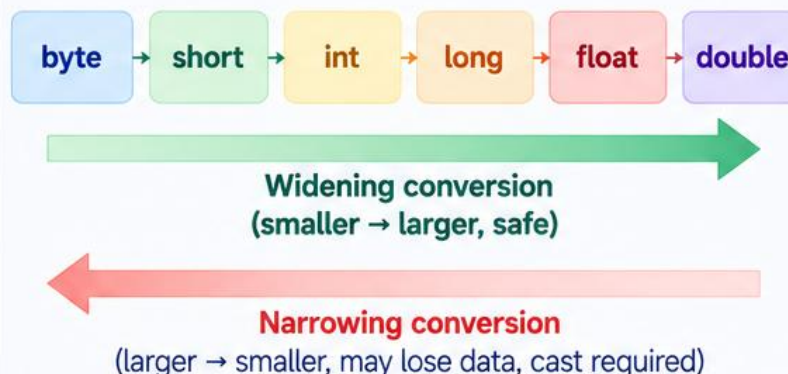
1 Type Conversion

Type conversion means changing a value from one data type to another compatible type.

- **Widening conversion (automatic)** — from a smaller type to a larger type. No data is lost.
- **Narrowing conversion (explicit)** — from a larger type to a smaller type. Data may be lost, so casting is required.

2 Numeric Type Hierarchy (Widening)

Automatic conversion in this direction (no cast needed)



3 Code Examples

Widening conversion (automatic)

```
int age = 19;
double d = age; // int → double
System.out.println(d); // 19.0
```



No cast is required.
The value is preserved.

Narrowing conversion (casting)

```
double grade = 87.8;
int wholeGrade = (int) grade; // 87
System.out.println(wholeGrade);
```



Cast is required.
The fractional part is lost.

4 Example: Student Grades

```
// Original value
double grade = 87.8;

// Widening: double → int is not automatic,
// but int → double is automatic
int age = 20;
double ageInDouble = age; // 20.0

// Narrowing: double → int (explicit cast)
int wholeGrade = (int) grade; // 87
System.out.println(wholeGrade);

// Another example
long students = 250;
int count = (int) students; // OK, value fits
```

5 Possible Loss of Data

Original value (double)	After casting to int	Result
87.8	(int) 87.8	87
65.0	(int) 65.0	65
-12.9	(int) -12.9	-12
123.999	(int) 123.999	123



When converting from a larger type to a smaller type, part of the value can be lost (e.g. fractional part).

6 Key Points

- 1 Widening conversion is automatic and safe.
- 2 Narrowing conversion requires an explicit cast.
- 3 Casting can lead to loss of data.
- 4 Use the appropriate type for your needs.
- 5 Check ranges when casting to avoid overflow.



Key idea: Choose the right data type and understand how conversion works to avoid unexpected results. Use casting only when necessary and be aware of possible data loss.

"Different types, same value — but not always the same result."



1 Widening Conversion (Automatic)

- From a smaller type to a larger type.
- Java converts automatically.
- No data is lost.
- Example: `int` → `double`.

```
int age = 20;  
double value = age; // automatic  
System.out.println(value);
```

Output:

20.0

20 → 20.0

byte → short → int → long → float → double

Widening
(smaller → larger, automatic)

The value was 20. After conversion, it became 20.0



Key idea: Use widening when possible (it is automatic and safe).
Use casting only when necessary, and be aware of possible data loss.

2 Narrowing Conversion (Casting)

- From a larger type to a smaller type.
- We must specify the new type (cast).
- Some data can be lost.
- Example: `double` → `int`.

```
double grade = 87.8;  
int wholeGrade = (int) grade; // cast  
System.out.println(wholeGrade);
```

Output:

87

87.8 → 87

87.8
(double)



87
(int)

Narrowing
(larger → smaller, cast required)

The value was 87.8. After conversion, it became 87

So, the .8 is lost here.



1 What is var?

- **var** is a local variable type inference feature introduced in Java 10.
- The compiler determines the actual type from the initializer (the value on the right side).
- It reduces boilerplate code and makes the program easier to read.
- Java is still **statically typed** — the type is known during compilation, not at runtime.

3 Example in a Student Application

```
public class StudentApp {  
    public static void main(String[] args) {  
        // Using var (Java 10+)  
        var studentName = "Anna";  
        var age = 19;  
        var grade = 87.5;  
        var passed = grade >= 50.0;  
        System.out.println(studentName + " passed: " + passed);  
    }  
}
```

4 When to use var?

- ✓ When the type is clear from the initializer.
- ✓ For local variables to reduce code verbosity.
- ✓ Especially useful with generic types, e.g. collections:

```
var students = new ArrayList<Student>();
```

2 Explicit Type vs. var

Both examples are equivalent. The compiler infers the same type.

Explicit type (traditional way)

```
String studentName = "Anna";  
int age = 19;  
double grade = 87.5;  
boolean enrolled = true;
```



Using var (type inference)

```
var studentName = "Anna";  
var age = 19;  
var grade = 87.5;  
var enrolled = true;
```

Variable	Value	Type inferred by the compiler
studentName	"Anna"	String
age	19	int
grade	87.5	double
enrolled	true	boolean



Important:

- ✓ **var** can be used only for **local** variables (inside methods).
- ✓ You must provide an initializer (a value).
- ✓ Java remains **statically typed** — the inferred type is fixed at compile time.

We write var, and the compiler determines the data type.

5 When not to use var?

- ✗ When the type is not obvious.
- ✗ For fields (class variables) — not allowed.
- ✗ For method parameters — not allowed.
- ✗ When it can reduce code readability.

6 Summary

- ✓ **var** makes code shorter and cleaner.
- ✓ The compiler infers the exact type.
- ✓ There is no performance difference.
- ✓ **Java is still a statically typed language.**



Use arithmetic operators to process sensor data and get useful results.

We use arithmetic operators to perform mathematical calculations.

+ is addition, - is subtraction, * is multiplication, / is division, and % gives the remainder.

For example, we can calculate the average temperature.

We can calculate the difference between two sensor readings.

We can also convert Celsius to Fahrenheit.

When we divide int by int, the result is an integer.

For example, 7 / 3 gives 2, not 2.33.

We cannot divide an integer by zero. It causes an error.

1 Arithmetic Operators in Java

Operator	Name	Example	Result
+	addition	5 + 3	8
-	subtraction	5 - 3	2
*	multiplication	5 * 3	15
/	division	10 / 3	3
%	remainder (modulo)	10 % 3	1
+ (unary)	unary plus	+5	5
- (unary)	unary minus	-5	-5
++	increment	x++	x = x + 1
--	decrement	x--	x = x - 1

2 Example 1: Average Temperature (+ and /)

We have temperature data from three rooms. We calculate the average.

```
double t1 = 22.5; // Room 1
double t2 = 23.0; // Room 2
double t3 = 21.5; // Room 3

double average = (t1 + t2 + t3) / 3;
System.out.println("Average temperature: " + average);
```

Output:

Average temperature: 22.3333333333



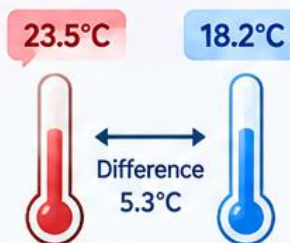
3 Example 2: Difference Between Readings (-)

We calculate the difference between two temperature readings.

```
double t1 = 23.5;
double t2 = 18.2;
double difference = t1 - t2;
System.out.println("Difference: " + difference);
```

Output:

Difference: 5.3



4 Example 3: Convert °C to °F (* and +)

Formula: $F = C \times 9 / 5 + 32$

```
double celsius = 25.0;
double fahrenheit = celsius * 9 / 5 + 32;
System.out.println("Temperature in °F: " + fahrenheit);
```

Output:

Temperature in °F: 77.0



5 Integer Division: int / int

When we divide two integers, the result is also an integer (fractional part is lost).

```
int a = 7;
int b = 3;
int result = a / b; // 7 / 3 = 2 (not 2.33)
System.out.println(result);
```

Output:

2

6 Division by Zero

Division by zero causes a runtime error.

```
int a = 10;
int b = 0;
int result = a / b;
```

Exception:

ArithmeticException:
/ by zero

7 Key Points

- ✓ Use +, -, *, /, % for basic calculations.
- ✓ Use unary + and - for sign.
- ✓ Use ++ and -- to increase or decrease a value.
- ✓ int / int gives an integer result.
- ✓ Do not divide by zero.
- ✓ Use arithmetic operators to process real data, e.g. sensor readings.



Update values.
Use compound operators.
Make code shorter.

The assignment operator = stores a value in a variable.

1 Assignment Operator (=)

The assignment operator stores a value in a variable.

```
int count = 0;           // store 0 in count
double total = 0.0;       // store 0.0 in total
double temperature = 22.5; // store 22.5
```



The value on the right side is calculated first.
Then the result is saved in the variable on the left side.

2 Compound Assignment Operators

A short way to calculate and save a new value.

Operator	Example	Equivalent Code	Meaning
<code>+=</code>	<code>total += temperature;</code>	<code>total = total + temperature;</code>	add and assign
<code>-=</code>	<code>total -= 5;</code>	<code>total = total - 5;</code>	subtract and assign
<code>*=</code>	<code>value *= 2;</code>	<code>value = value * 2;</code>	multiply and assign
<code>/=</code>	<code>value /= 4;</code>	<code>value = value / 4;</code>	divide and assign
<code>%=</code>	<code>value %= 3;</code>	<code>value = value % 3;</code>	remainder and assign

For example,
`temperature = 22.5;`
stores 22.5 in
temperature.

3 Example: Processing Temperature Readings

We add each sensor value to the total and increase the count.

```
double total = 0.0;
int count = 0;

double temperature = 22.5; // new reading
total += temperature;      // total = total + temperature
count++;                  // count = count + 1

temperature = 23.0;        // next reading
total += temperature;
count++;
```

Temperature sensor



Update values

```
total += 22.5;
count++;
```

4 Before and After

We calculate a new value and save it in the variable.

Step	Code	Explanation	Result
1	<code>double total = 0.0;</code> <code>double temperature = 22.5;</code>	Initial values	<code>total = 0.0</code>
2	<code>total = total + temperature;</code> <code>// or: total += temperature;</code>	Calculate and save (add the value)	<code>total = 22.5</code>
3	<code>int count = 0;</code> <code>count++;</code>	Increase by 1	<code>count = 1</code>
Example	<code>count += 5;</code>	Add 5 and save (same as <code>count = count + 5</code>)	<code>count = 6</code>

Compound assignment operators calculate and save a new value.

For example, `total += temperature;` means
`total = total + temperature;`

The value of total is updated.

5 Difference: Calculation vs. Saving

Calculation (without saving)

The expression is calculated, but the result is not stored.

```
double result = total + temperature;
// total is not changed
```

Calculation and saving

The new value is calculated and stored in the same variable.

```
total += temperature;
// total is updated
```

6 Real-World Example: Air Quality Monitoring

```
double totalTemp = 0.0;
int count = 0;

// new sensor reading
double temperature = 21.8;
totalTemp += temperature; // add to total
count++;                  // increase count
```



7 Key Points

- ✓ = stores a value in a variable.
- ✓ Use compound operators to write shorter code.
- ✓ The right side is calculated first.
- ✓ The result is saved in the variable.
- ✓ Useful for processing sensor data (sum, count, average, etc.).

`count++` increases the value of count by one.



1 Arithmetic operators in Java

Java provides arithmetic operators for numeric calculations.

Operator	Name	Example	Result
+	Addition	5 + 3	8
−	Subtraction	5 − 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 3	3 (integer) 3.33 (double)
%	Remainder (modulo)	10 % 3	1

2 Algebraic problem

A teacher gives the following task to students:

"Calculate the value of the function

$$y = 2x^2 + 3x - 5$$

for a given x."

Students must implement the formula in a Java program and find the result for different values of x.

3 Java implementation

```
// Calculate y = 2x^2 + 3x - 5
public class AlgebraExample {
    public static void main(String[] args) {
        double x = 4.0;           // given value
        double y = 2 * x * x + 3 * x - 5;
        System.out.println("x = " + x);
        System.out.println("y = " + y);
    }
}
```

Output (for x = 4.0)

```
x = 4.0
y = 43.0
```

4 Step by step calculation (x = 4)

$$y = 2x^2 + 3x - 5$$

- 1 $x = 4$
- 2 $x^2 = 4^2 = 16$
- 3 $2x^2 = 2 * 16 = 32$
- 4 $3x = 3 * 4 = 12$
- 5 $y = 32 + 12 - 5 = 39$



The same formula can be used for any value of x.

Try x = 1, 2, 4, 10 and check the results!

5 Real-world connection

In the educational process:



Teacher
gives a mathematical problem or formula.



Student
implements the formula in Java, tests it with different values and analyzes the result.



Result
Students develop programming skills and apply mathematics in practice.

6 Key takeaways

- ✓ Arithmetic operators help us solve real problems.
- ✓ We can implement algebraic formulas directly in Java.
- ✓ The same code works for different input values.
- ✓ Division of integers gives an integer result. Use double for decimal results.
- ✓ The % operator is useful for tasks like checking if a number is even or odd.

Practical Task: Solve a Quadratic Equation

Quadratic equation: step by step in Java



A quadratic equation has the form $ax^2 + bx + c = 0$, where a , b and c are real numbers and $a \neq 0$.

1 Formula

$$ax^2 + bx + c = 0$$

Discriminant:

$$D = b^2 - 4ac$$

Number of solutions:

- $D > 0$ — two real solutions
- $D = 0$ — one real solution
- $D < 0$ — no real solutions

$$x_1 = \frac{-b + \sqrt{D}}{2a} \quad x_2 = \frac{-b - \sqrt{D}}{2a}$$

2 Input from the user

Use `Scanner` to read the coefficients a , b and c from the keyboard.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter a: ");
double a = sc.nextDouble();
System.out.print("Enter b: ");
double b = sc.nextDouble();
System.out.print("Enter c: ");
double c = sc.nextDouble();
```

3 Program code (Main.java)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a: ");
        double a = sc.nextDouble();
        System.out.print("Enter b: ");
        double b = sc.nextDouble();
        System.out.print("Enter c: ");
        double c = sc.nextDouble();

        double D = b * b - 4 * a * c;

        if (D > 0) {
            double x1 = (-b + Math.sqrt(D)) / (2 * a);
            double x2 = (-b - Math.sqrt(D)) / (2 * a);
            System.out.println("Two solutions:");
            System.out.println("x1 = " + x1);
            System.out.println("x2 = " + x2);
        } else if (D == 0) {
            double x = -b / (2 * a);
            System.out.println("One solution:");
            System.out.println("x = " + x);
        } else {
            System.out.println("No real solutions (D < 0).");
        }
    }
}
```

4 Example run

Input:

```
Enter a: 1
Enter b: -5
Enter c: 6
```

Output:

```
Two solutions:
x1 = 3.0
x2 = 2.0
```

5 Try it yourself

1. Run the program for different values and check the results.
2. Test these cases:
 - two solutions: $a = 1$, $b = -5$, $c = 6$
 - one solution: $a = 1$, $b = 4$, $c = 4$
 - no solutions: $a = 1$, $b = 2$, $c = 5$



Key Takeaways

- Use the discriminant $D = b^2 - 4ac$.
- If $D > 0$ — two real solutions.
- If $D = 0$ — one real solution.
- If $D < 0$ — no real solutions.
- Use `Math.sqrt(D)` to calculate the square root.
- `Scanner` is used to read input from the user.



Compare sensor data and use logical conditions to detect problems.

1 Comparison Operators

We use comparison operators to compare two values. The result is always a boolean value: **true** or **false**.

Operator	Name	Example	Result
>	greater than	25 > 20	true
<	less than	18 < 20	true
>=	greater than or equal to	20 >= 20	true
<=	less than or equal to	18 <= 20	true
==	equal to	22 == 22	true
!=	not equal to	22 != 20	true

2 Logical Operators

We use logical operators to combine conditions. The result is also a boolean value: **true** or **false**.

Operator	Name	Example	Result
&&	AND (both conditions must be true)	temperature > 25 && humidity > 60	true (if both are true)
	OR (at least one condition is true)	temperature > 25 CO2 > 800	true (if any is true)
!	NOT (inverts the result)	!isSafe	true (if isSafe is false)

3 Real-World Example: Air Quality Monitoring

We check sensor values and use logical operators to make a decision.

```
double temperature = 26.5;
int co2 = 900;

if (temperature > 25 && co2 > 800)
    System.out.println("Warning:
        Poor air quality!");
} else {
    System.out.println("Air quality
        is normal.");
}
```

4 Examples with Sensor Data

A. Single Comparison

Check if temperature is high.

```
double temperature = 28.0;
if (temperature > 25) {
    System.out.println("High
        temperature!");
}
```

Output:
High temperature!



28.0°C

B. Using AND (&&)

Check if temperature and humidity are high at the same time.

```
double temperature = 26.0;
int humidity = 70;

if (temperature > 25 && humidity > 60) {
    System.out.println("Hot and humid!");
} else {
    System.out.println("Normal.");
}
```

Output:
Hot and humid!



26.0°C



70%

C. Using OR (||)

Check if CO₂ is high or PM2.5 is high.

```
int co2 = 850;
int pm25 = 45;

if (co2 > 800 || pm25 > 35) {
    System.out.println("Poor
        air quality!");
} else {
    System.out.println("Air
        quality is good.");
}
```

Output:
Poor air quality!



850 ppm



PM2.5
45 µg/m³

5 Key Points

- 1 Comparison operators (>, <, >=, <=, ==, !=) return **true** or **false**.
- 2 Logical operators (&&, ||, !) combine conditions.
- 3 We use these operators in **if** statements to make decisions.
- 4 Both operands of a comparison must be compatible types.
- 5 Logical operators are very useful for real-world applications, especially when checking sensor data.

We use comparison operators to compare values.

The result is always true or false.

For example, 28 > 25 is true.

We use logical operators to combine conditions.

&& means AND. Both conditions must be true.

**`

! means NOT. It changes true to false and false to true.

We use these operators to make decisions in a program.

```
temperature > 25 && CO2 > 800
```

The program checks two conditions: Is the temperature above 25?

Is CO₂ above 800?

If both conditions are true, the program shows a warning.



Evaluate complex conditions in the Air Quality Monitoring System correctly.

Java evaluates operators in a specific order.

1 What is operator precedence?

- When an expression has multiple operators, the operators are evaluated in a specific order (precedence).
- Operators with **higher precedence** are evaluated first.
- We can use **parentheses ()** to change the order and make the expression clear.

2 Precedence (main operators)

Priority	Operators	Example
1	()	(a + b) * c
2	unary +, -, !	-a, !isSafe
3	* / %	a * b
4	+ -	a + b
5	> < >= <=	a > b
6	== !=	a == b
7	&&	a > 25 && b > 800
8		a > 25 b > 800
9	=, +=, -=, ...	a = b

Higher precedence



Lower precedence

3 Using parentheses

Parentheses change the evaluation order.

```
boolean alert = temperature > 25
|| CO2 > 800 && humidity > 60;
```

&& has higher precedence than ||

This code is evaluated as:

```
temperature > 25 || (CO2 > 800 && humidity > 60)
```

```
boolean alert = (temperature > 25 || CO2 > 800)
&& humidity > 60;
```

Parentheses make the condition clear:

```
(temperature > 25 || CO2 > 800) && humidity > 60
```

Operators with higher priority are evaluated first.

We use parentheses () to change or clarify the order.

For example, multiplication is performed before addition.

****&& means AND, and `**

With &&, if the first condition is false, Java does not check the second condition.

****With `**

This is called short-circuit evaluation.

4 Short-circuit evaluation

For logical operators **&&** and **||**, Java does not always evaluate the second condition.

&& (AND)

If the first condition is **false**, the second condition is **not** evaluated.

false && ...

✗ second condition is skipped

|| (OR)

If the first condition is **true**, the second condition is **not** evaluated.

true || ...

✗ second condition is skipped

5 Example: Air Quality Monitoring

We check several sensor values and send an alert if needed.

```
double temperature = 27.5; // °C
int co2 = 900; // ppm
int pm25 = 45; // µg/m³
boolean doorOpen = false;
```

```
if ((temperature > 25 || co2 > 800) && pm25 > 35
&& !doorOpen) {
    System.out.println("Warning: Poor air quality!");
} else {
    System.out.println("Air quality is normal.");
}
```



temperature > 25

27.5 > 25 ✓ true



co2 > 800

900 > 800 ✓ true



pm25 > 35

45 > 35 ✓ true



!doorOpen

!false ✓ true

All conditions are true → **alert**

So, Java evaluates operators in a specific order.

We use parentheses to control this order.

With **&&** and **||**, Java may not check the second condition.

7 Key Points

- ✓ Java evaluates operators according to their precedence.
- ✓ Use parentheses to make complex conditions clear.
- ✓ **&&** and **||** use short-circuit evaluation (the second condition may not be evaluated). This helps to avoid unnecessary calculations and possible errors.
- ✓ We use these operators to check sensor values and make decisions in the Air Quality Monitoring System.



1 Wrapper Classes in Java

For each primitive type Java provides a wrapper class in the java.lang package.

Primitive type	Wrapper class	Example value
byte	Byte	Byte b = 10;
short	Short	Short s = 20;
int	Integer	Integer i = 100;
long	Long	Long l = 1000L;
float	Float	Float f = 3.14f;
double	Double	Double d = 87.5;
char	Character	Character c = 'A';
boolean	Boolean	Boolean b = true;

4 Why Do We Need Wrapper Types?

- Wrapper classes are objects, so they can be used in Collections (List, Set, Map, etc.).
- They provide useful methods (parseXxx, valueOf, compareTo).
- They allow null values (unlike primitive types).
- They are required in APIs and generic classes.



2 Primitive ↔ Wrapper Conversion

Autoboxing (automatic)

Primitive type is automatically converted to a wrapper object.

```
int age = 19;
Integer ageObj = age; // autoboxing
System.out.println(ageObj); // 19
```

Unboxing (automatic)

Wrapper object is automatically converted to a primitive type.

```
Integer countObj = 25;
int count = countObj; // unboxing
System.out.println(count); // 25
```

3 Autoboxing and Unboxing with Methods

Explicit boxing and unboxing

```
Integer numObj = Integer.valueOf(42);
// boxing (explicit)
int num = numObj.intValue();
// unboxing (explicit)
```

Parsing from String

```
String s1 = "25";
int value1 = Integer.parseInt(s1);
String s2 = "87.5";
double value2 = Double.parseDouble(s2);
boolean flag = Boolean.parseBoolean("true");
```

5 Example: Using in Collections

```
import java.util.*;

List<Integer> scores = new ArrayList<>();
scores.add(95); // autoboxing
scores.add(87);
scores.add(100);
```

```
for (Integer s : scores) {
    System.out.println(s); // unboxing
}
```

Memory view

95

87

100

List<Integer>

Stores Integer objects

6 Key Takeaways

- ✓ Wrapper classes provide object representations of primitive types.
- ✓ Autoboxing and unboxing work automatically.
- ✓ Wrapper types are necessary for Collections.
- ✓ Wrapper classes have useful static methods (e.g. parseInt, parseDouble).
- ✓ Wrapper types can hold null values.
- ✓ Use primitives for performance, but wrappers when you need objects (Collections, generics, APIs).

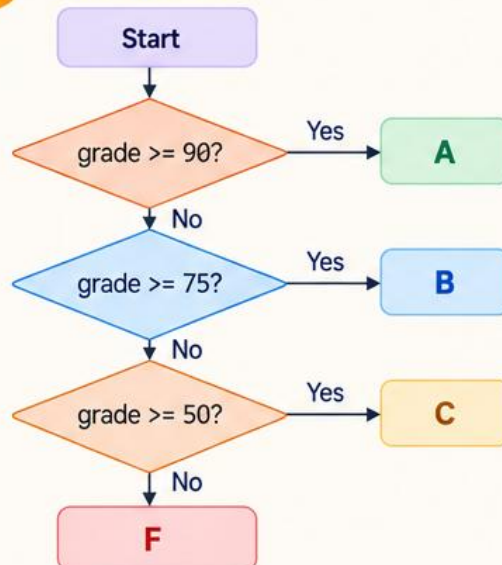


1 Problem: Determine the Result

The teacher defines the result based on the grade:

Grade range	Result (letter)	Description
grade >= 90	A	Excellent
grade >= 75	B	Good
grade >= 50	C	Satisfactory
otherwise	F	Fail

4 Flowchart



2 Solution 1: if – else if

Traditional approach using if–else if–else.

```

public class GradeExampleIf {
    public static void main(String[] args) {
        double grade = 87.5;
        String result;
        if (grade >= 90) {
            result = "A";
        } else if (grade >= 75) {
            result = "B";
        } else if (grade >= 50) {
            result = "C";
        } else {
            result = "F";
        }
        System.out.println("Grade: " + grade);
        System.out.println("Result: " + result);
    }
}
  
```

5 Examples of Results

Grade (input)	if – else if	switch expression
95	A	A
80	B	B
65	C	C
40	F	F

3 Solution 2: Modern switch Expression

Using the switch expression (Java 14+).

```

public class GradeExampleSwitch {
    public static void main(String[] args) {
        double grade = 87.5;
        String result = switch ((int) grade) {
            case 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
                -> "A";
            case 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89
                -> "B";
            case 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64,
                65, 66, 67, 68, 69, 70, 71, 72, 73, 74
                -> "C";
            default -> "F";
        };
        System.out.println("Grade: " + grade);
        System.out.println("Result: " + result);
    }
}
  
```

6 Program Output

```

Grade: 87.5
Result: B

Grade: 95.0
Result: A

Grade: 40.0
Result: F
  
```

7 Key Takeaways

- ✓ Use if – else if for flexible conditions.
- ✓ Use switch expression in modern Java for clear and concise code.
- ✓ Both approaches produce the same result.
- ✓ Switch expressions return a value.
- ✓ This example can be used in many real tasks, e.g. grading students.



1 Array of Student Grades

We store grades of a group in an array.

```
int[] grades = {85, 92, 76, 88, 95};
System.out.println("Number of students: "
    + grades.length); // 5
```

Index:	0	1	2	3	4
grades:	85	92	76	88	95



An array stores multiple values of the same type.
Array indices start from 0.

2 Processing with for Loop

Calculate sum, average, minimum and maximum grade.

```
int[] grades = {85, 92, 76, 88, 95};
int sum = 0;
int min = grades[0];
int max = grades[0];

for (int i = 0; i < grades.length; i++) {
    int g = grades[i];
    sum += g;
    if (g < min) min = g;
    if (g > max) max = g;
}

double average = (double) sum / grades.length;
System.out.println("Sum: " + sum);
System.out.println("Average: " + average);
System.out.println("Min: " + min);
System.out.println("Max: " + max);
```

3 Using Enhanced for Loop

Count how many students passed the exam (grade ≥ 50).

```
int[] grades = {85, 92, 76, 88, 95};
int passedCount = 0;

for (int grade : grades) { // enhanced for
    if (grade >= 50) {
        passedCount++;
    }
}

System.out.println("Passed: " + passedCount);
System.out.println("Not passed: "
    + (grades.length - passedCount));
```



Enhanced for loop

- ✓ Simpler and more readable.
- ✓ We get each element directly, without using the index.

4 Using while Loop

The same calculation using a while loop.

```
int[] grades = {85, 92, 76, 88, 95};
int i = 0;
int sum = 0;

while (i < grades.length) {
    sum += grades[i];
    i++;
}

double average = (double) sum / grades.length;
System.out.println("Average (while): " + average);
```

5 Example Output

```
Number of students: 5
Sum: 436
Average: 87.2
Min: 76
Max: 95

Passed: 5
Not passed: 0
Average (while): 87.2
```

6 Results and Analysis

Student	Grade	Status (grade ≥ 50)
1	85	Passed
2	92	Passed
3	76	Passed
4	88	Passed
5	95	Passed

Count	Sum	Average	Min	Max	Passed	Not passed
5	436	87.2	76	95	5	0



Key point: Arrays and loops allow us to process many values. We can calculate statistics (sum, average, min, max) and make decisions, for example, how many students passed the exam.

"One array, many results."



1

Student data

Name, group,
grades



2

Data types

String, int, double,
char, boolean

```
String name
int age
double grade
char group
boolean passed
```

3

Array of grades

Store multiple
grades

grades

85	92	76	88	95
----	----	----	----	----

4

Arithmetic operators

sum, average,
min, max



5

Comparison and logic

>=, <=, ==
&&, ||, !



6

if / switch

Determine
letter grade

```
>= 90 -> A
>= 75 -> B
>= 50 -> C
else -> F
```

7

Final result

Student report
and statistics



8

Complete Java Program

```
public class StudentPerformance {
    public static void main(String[] args) {
        String[] names = {"Anna", "Bekzat", "Diana", "Ermek", "Sara"};
        char group = 'A';
        int[] grades = {85, 92, 76, 88, 95};

        int sum = 0;
        int min = grades[0];
        int max = grades[0];
        int passedCount = 0;

        System.out.println("Student Grades (Group " + group + "):");
        System.out.println("-----");

        for (int i = 0; i < grades.length; i++) {
            int grade = grades[i];
            sum += grade;
            if (grade < min) min = grade;
            if (grade > max) max = grade;

            String letter = getLetterGrade(grade); // using switch
            boolean passed = grade >= 50;
            if (passed) passedCount++;

            System.out.printf("%-8s  %5d  %5s  %s\n", names[i], grade, letter,
                             passed ? "Passed" : "Not passed");
        }

        double average = (double) sum / grades.length;

        System.out.println("Number of students: " + grades.length);
        System.out.println("Sum: " + sum + ", Average: " + average);
        System.out.println("Min: " + min + ", Max: " + max);
        System.out.println("Passed: " + passedCount + ", Not passed: " + (grades.length - passedCount));
    }
}
```

9

Program Output (example)

Student Grades (Group A)

```
-----
Anna      85      B      Passed
Bekzat    92      A      Passed
Diana     76      B      Passed
Ermek     88      B      Passed
Sara      95      A      Passed
-----
```

```
Number of students: 5
Sum: 436, Average: 87.20
Min: 76, Max: 95
Passed: 5, Not passed: 0
```

10

Explanation

Concept	Where in the program
Student data	String[] names, char group
Data types	String, int, double, char, boolean
Array of grades	int[] grades, grades.length
Arithmetic operators	sum += grade, average, min, max
Comparison operators	grade >= 50, grade < min, grade > max
Logical operators	passed ? .., (grade >= 50)
if statement	if (grade >= 50) passedCount++
switch expression	getLetterGrade(int grade)
Loops	for (int i = 0; i < grades.length; i++)
Final result	Table of students and statistics

11

Practice Task



- 1 Change the program for your group of students (5-10 students).
- 2 Use your own names and grades.
- 3 Show the result in the same format.
- 4 Add one more statistic, for example, the number of students with grade A.
- 5 (Optional) Read the number of students and their grades from the keyboard using Scanner.

12

Key Takeaways

- ✓ We can process real data using arrays and loops.
- ✓ We use arithmetic, comparison and logical operators to analyze data.
- ✓ We can make decisions with if or switch.
- ✓ We get useful results: average, min, max and the number of passed students.
- ✓ All main concepts from this lecture are combined in one complete program.

22 if Statement (Simple Condition)



The **if** statement allows a block of code to be executed if a condition is **true**. If the condition is **false**, the block of code is skipped.

1 Syntax

```
if (condition) {  
    // code to execute  
    // if the condition is true  
}
```

- The condition is a logical expression written in parentheses ().
- If the condition is **true**, the block of code { ... } is executed.
- If the condition is **false**, the block of code is skipped.

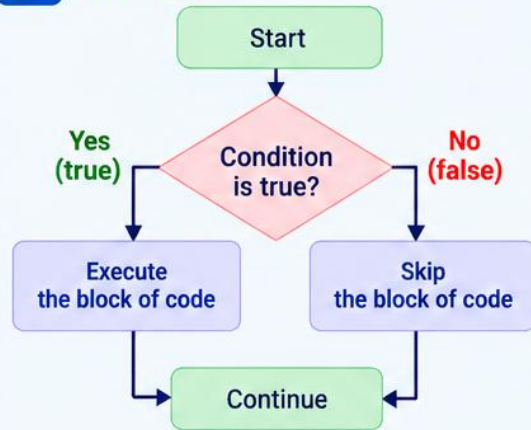
2 Logical Expression

A condition is an expression that returns a **boolean** value (**true** or **false**).

Examples of conditions:

- $x > 0$
- $\text{age} \geq 18$
- $a == b$
- $\text{number} \% 2 == 0$

3 Flowchart



4 Example 1. Checking a Number

```
public class Example1 {  
    public static void main(String[] args) {  
        int number = 10;  
        if (number > 0) {  
            System.out.println("Positive number");  
        }  
    }  
}
```

Output:

Positive number

5 Task. Age Check

Task: Given an integer number — a person's age. If the age is 18 or greater, print "Access granted", otherwise print "Access denied".

```
import java.util.Scanner;  
  
public class AgeCheck {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter age: ");  
        int age = sc.nextInt();  
  
        if (age >= 18) {  
            System.out.println("Access granted");  
        } else {  
            System.out.println("Access denied");  
        }  
    }  
}
```

Example run:

Enter age: 20
Access granted

Enter age: 15
Access denied

6 Real-life Example



A system checks the age of a visitor. If the age is 18 or older — access is allowed, otherwise access is prohibited. This is a simple example of using the **if** statement in real life.

7



Key Takeaways

- The **if** statement executes code only when the condition is **true**.
- A condition is a logical expression of type **boolean**.
- If the condition is **false**, the block of code is skipped.
- The **if** statement is used for decision making in a program.



Try It Yourself

Modify the program:

- Check if a number is even ($\text{number} \% 2 == 0$).
- Print "Even number" if the condition is true.
- Test the program with different numbers.

Small steps
— big results!

23

if – else if – else Statement (Multiple Alternatives)



The **if – else if – else** statement allows you to choose **one of several** blocks of code depending on the condition. Only the **first** matching block is executed.

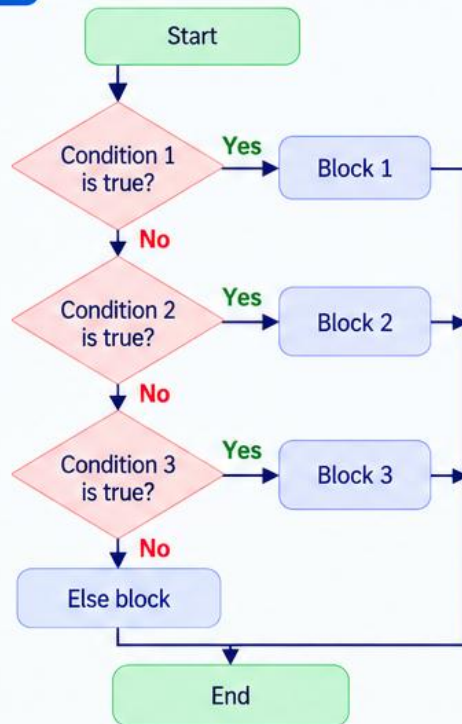
1 Syntax

```
if (condition1) {
    // block 1
} else if (condition2) {
    // block 2
} else if (condition3) {
    // block 3
} else {
    // default block
}
```

2 How It Works

- Conditions are checked from top to bottom.
- The **first** block with a **true** condition is executed.
- The remaining blocks are skipped.
- The **else** block is executed if none of the conditions are true.

3 Flowchart



4 Example Code

```
public class GradeExample {
    public static void main(String[] args) {
        int score = 85;
        String grade;

        if (score >= 90) {
            grade = "A";
        } else if (score >= 80) {
            grade = "B";
        } else if (score >= 70) {
            grade = "C";
        } else if (score >= 60) {
            grade = "D";
        } else {
            grade = "F";
        }

        System.out.println("Score: " + score);
        System.out.println("Grade: " + grade);
    }
}
```

5 Task

Task: A student's grade (an integer from 0 to 100) is given. Output the letter grade according to the following scale:

Score Range	Grade
90 – 100	A
80 – 89	B
70 – 79	C
60 – 69	D
0 – 59	F

Example input:

85

Example output:

Score: 85

Grade: B

6 Real-life Example



A system displays a student's grade based on the total score. It uses several conditions to determine the result. This is a practical example of using the **if – else if – else** statement in real life.



Key Takeaways

- The **if – else if – else** statement allows you to choose one of several alternatives.
- Conditions are checked in order from top to bottom.
- Only the first matching block is executed.
- The **else** block is used when none of the conditions are true.
- It is commonly used for decision making in programs.



Try It Yourself

Modify the program:

- Test with different scores (e.g. 95, 78, 60, 45).
- Add a message: "Excellent!", "Good!", "Average", "You need to study more".
- Use an integer variable for the score.
- You can also use your own grading scale.



The switch statement allows us to choose one of several code blocks depending on the value of an expression. It is a convenient alternative to multiple if-else statements.

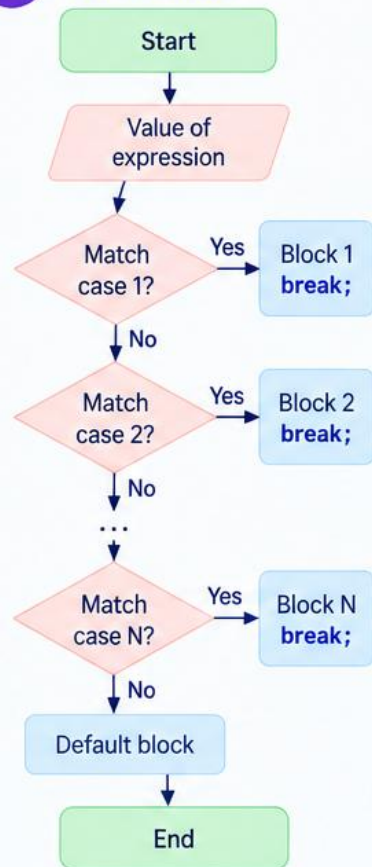
1 Syntax

```
switch (expression) {
    case value1:
        // code block 1
        break;
    case value2:
        // code block 2
        break;
    ...
    default:
        // code block if no match
}
```

2 How it works

- 1 The expression is evaluated once.
- 2 Its value is compared with the values in each case.
- 3 If a match is found, the corresponding code block is executed.
- 4 The **break** statement stops the execution of the switch.
- 5 The **default** block is executed if there is no match.

3 Flowchart



4 Example: Day of the Week

Task: Given a number from 1 to 7, print the name of the day. If the number is outside the range, print an error message.

```
import java.util.Scanner;

public class SwitchExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number (1-7): ");
        int day = sc.nextInt();

        String dayName;
        switch (day) {
            case 1:
                dayName = "Monday";
                break;
            case 2:
                dayName = "Tuesday";
                break;
            case 3:
                dayName = "Wednesday";
                break;
            case 4:
                dayName = "Thursday";
                break;
            case 5:
                dayName = "Friday";
                break;
            case 6:
                dayName = "Saturday";
                break;
            case 7:
                dayName = "Sunday";
                break;
            default:
                dayName = "Invalid number!";
        }
        System.out.println("Day: " + dayName);
    }
}
```

5 Program output examples

Example 1 (correct input):

Enter a number (1-7): 3
Day: Wednesday

Example 2 (incorrect input):

Enter a number (1-7): 9
Day: Invalid number!

7 Real-life example



A similar choice is used in real applications, for example, in calendars, menus, ATMs, booking systems, games, etc.

6 Key Points

- ✓ The switch statement is convenient when we need to choose one option from many.
- ✓ Do not forget the **break** statement to avoid executing the next cases.
- ✓ The **default** block handles all other values.
- ✓ The switch statement works with primitive types (int, char, enum) and some reference types (String) in modern Java versions.

25

for Loop (a Known Number of Repetitions)



The **for** loop is used when the number of repetitions is known in advance.

1 Syntax

```
for (initialization; condition;
    update) {
    // code block
}
```

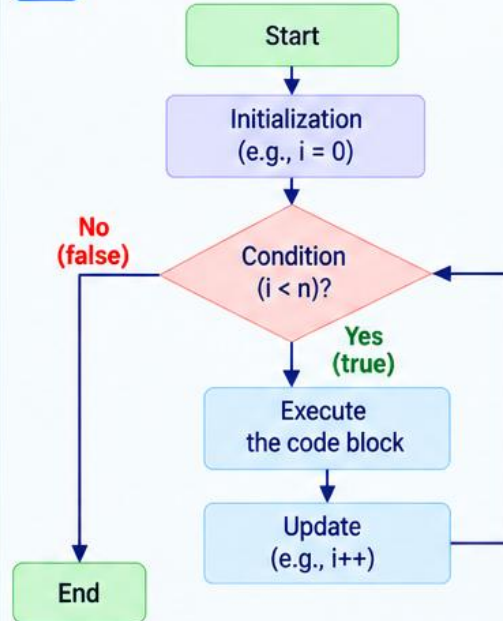
Example:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

2 How It Works

- 1 The **initialization** (usually sets the initial value of the counter).
- 2 The condition is checked.
- 3 If the condition is **true**, the code block is executed.
- 4 The **update** (usually increases or decreases the counter).
- 5 Return to step 2.
- 6 The loop continues until the condition becomes **false**, and then the loop terminates.

3 Flowchart



4 Example. Sum of the First n Numbers

Task: Given a natural number n . Calculate the sum of the first n natural numbers: $1 + 2 + \dots + n$.

```
import java.util.Scanner;

public class SumExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i; // sum = sum + i
        }

        System.out.println("Suma = " + sum);
    }
}
```

Example input:

Enter n: 5
Sum = 15

Explanation:

$1 + 2 + 3 + 4 + 5 = 15$

5 Another Example

Display the multiplication table for 3 (from 1 to 10) on the screen.

```
public class MultiplicationTable {
    public static void main(String[] args) {
        int n = 3;
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }
}
```

Result:

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```



Key Takeaways

- The **for** loop is used when the number of repetitions is known in advance.
- It consists of three parts: initialization, condition, update.
- It is often used for processing arrays, calculating sums, searching for maximum or minimum values, etc.
- The number of iterations can be any integer (increasing or decreasing the counter).

Same logic -
many
applications!



26 while and do-while Loops



While and do-while loops are used when the number of repetitions is not known in advance. The loop continues to execute as long as the condition is **true**.

1 Syntax

while loop

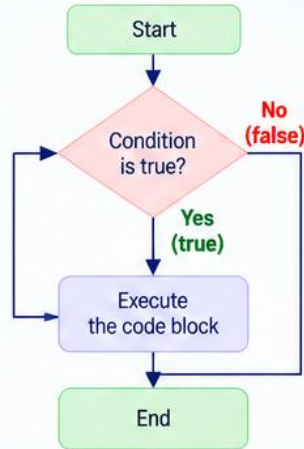
```
while (condition) {  
    // code block  
}
```

do-while loop

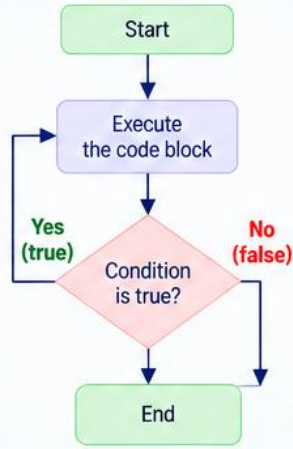
```
do {  
    // code block  
} while (condition);
```

3 Flowcharts

while loop



do-while loop



4 Example 1. while loop — sum of numbers (1..n)

Task: Enter a number n and calculate the sum of numbers from 1 to n using a while loop.

```
import java.util.Scanner;  
  
public class WhileSum {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter n: ");  
        int n = sc.nextInt();  
  
        int i = 1;  
        int sum = 0;  
        while (i <= n) {  
            sum += i;  
            i++;  
        }  
  
        System.out.println("Sum = " + sum);  
    }  
}
```

Example input/output:

Enter n: 5
Sum = 15

How it works?

- 1 i = 1, sum = 0
- 2 While i <= n, add i to the sum.
- 3 Increase i by 1.
- 4 When i > n, the loop terminates.

2 Main Differences

Feature	while	do-while
Condition check	Before executing the block	After executing the block
Minimum number of iterations	0 times	Always at least 1 time
When to use?	When the block may not be executed	When the block must be executed at least once

5 Example 2. do-while — input a positive number

Task: Keep asking the user to enter a number until a positive number (greater than 0) is entered. Use a **do-while** loop.

```
import java.util.Scanner;  
  
public class PositiveNumber {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int number;  
        do {  
            System.out.print("Enter a positive number: ");  
            number = sc.nextInt();  
        } while (number <= 0);  
  
        System.out.println("You entered: " + number);  
    }  
}
```

Example run:

Enter a positive number: -3
Enter a positive number: 0
Enter a positive number: 7
You entered: 7



6 Key Takeaways

- **while** and **do-while** loops are used when the number of repetitions is not known in advance.
- The loop may execute zero or more times (**while**) or at least once (**do-while**).
- Both use a boolean condition.
- They are commonly used for input validation, searching, and processing a sequence of data.



7 Try It Yourself

- Use a while loop to find the product of numbers from 1 to n.
- Use a do-while loop to ask for a password until the correct word "java" is entered.



The **break** and **continue** statements are used in loops to control their execution. **break** completely terminates the loop. **continue** skips the current iteration and moves to the next one.

1 Syntax

break

```
for (...) {
    if (condition) {
        break; // exit the loop
    }
}
```

continue

```
for (...) {
    if (condition) {
        continue; // skip the current
                  // iteration
    }
}
```

2 How It Works

- **break** immediately terminates the execution of the innermost loop and control is passed to the statement following the loop.
- **continue** skips the remaining statements in the current iteration and immediately moves to the next iteration.

3 Example 1. break — search for a number

Task: Find the first number greater than 50 in an array. As soon as the number is found, terminate the loop (use **break**).

```
public class BreakExample {
    public static void main(String[] args) {
        int[] numbers = {10, 25, 37, 42, 56, 68, 79};

        for (int num : numbers) {
            if (num > 50) {
                System.out.println("First number > 50: " + num);
                break; // exit the loop
            }
            System.out.println("Checking: " + num);
        }
        System.out.println("Search finished.");
    }
}
```

Output:

```
Checking: 10
Checking: 25
Checking: 37
Checking: 42
First number > 50: 56
Search finished.
```

After the first number greater than 50 is found, the loop terminates and the remaining elements are not checked.

4 Example 2. continue — skip even numbers

Task: Print only odd numbers from 1 to 10. Skip even numbers (use **continue**).

```
public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) {
                continue; // skip even numbers
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
3
5
7
9
```

Even numbers (2, 4, 6, 8, 10) are not printed, because the loop immediately moves to the next iteration.

5 Another Example: loops and break + continue

Task: Iterate through the numbers from 1 to 10. Skip the number 5 (use **continue**) and terminate the loop if the number is greater than 8 (use **break**).

```
public class BreakContinueDemo {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue; // skip 5
            }
            if (i > 8) {
                break; // exit the loop
            }
            System.out.println("i = " + i);
        }
        System.out.println("Loop finished.");
    }
}
```

Output:

```
i = 1
i = 2
i = 3
i = 4
i = 6
i = 7
i = 8
Loop finished.
```

What happens?

- When $i == 5$, it is skipped (**continue**).
- When $i > 8$ (i.e. 9), the loop terminates (**break**).
- The printed numbers are: 1, 2, 3, 4, 6, 7, 8.

6



Key Takeaways

- **break** terminates the loop.
- **continue** skips the current iteration and moves to the next one.
- Used in **for**, **while**, and **do-while** loops.
- Help control loop execution and make the code more flexible.

7



Try It Yourself

1. Find the first number divisible by 7 in the range from 1 to 100 (use **break**).
2. Print all numbers from 1 to 20 that are not divisible by 3 (use **continue**).
3. In a loop from 1 to 50, skip even numbers and terminate the loop when a number becomes greater than 30 (use both **break** and **continue**).
4. Process an array of integers: skip negative numbers, print positive numbers, and terminate the loop if a zero is encountered.

28

Arrays in Java – Creation and Initialization



An array is a data structure that stores multiple values of the same type. Array elements have indexes that start at 0.

1 Ways to Create an Array

1 Declaring an array (specifying the size)

```
int[] numbers = new int[5];
// array of 5 integers
```

0	1	2	3	4
0	0	0	0	0

2 Declaring and initializing with values

```
int[] numbers = {10, 20, 30, 40, 50};
```

0	1	2	3	4
10	20	30	40	50

3 Shorthand initialization

```
int[] numbers = new int[]{10, 20, 30};
// can be done without specifying the size
```

2 Accessing Elements

```
int[] numbers = {10, 20, 30, 40, 50};
System.out.println(numbers[0]); // 10
System.out.println(numbers[2]); // 30
numbers[4] = 100; // change value
```

3 Array Length

```
int[] numbers = {10, 20, 30};
System.out.println(numbers.length); // 3
```

4 Example 1. Finding the Maximum Value

Task: Given an array of integers, find and output the maximum value.

```
public class MaxInArray {
    public static void main(String[] args) {
        int[] numbers = {12, 7, 25, 3, 18};

        int max = numbers[0]; // assume the first element
                             // is the maximum

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] > max) {
                max = numbers[i];
            }
        }

        System.out.println("Maximum value: " + max);
    }
}
```

Output:

Maximum value: 25

5 Another Example. Average Value

Task: Given an array of numbers, find the average value.

```
public class AverageExample {
    public static void main(String[] args) {
        double[] values = {5.5, 3.0, 4.0, 2.5};
        double sum = 0;

        for (int i = 0; i < values.length; i++) {
            sum += values[i];
        }

        double average = sum / values.length;
        System.out.println("Average value: " + average);
    }
}
```

Output:

Average value:
3.75

6 Array of Strings

```
String[] names = {"Ana", "Bob", "Tom"};
System.out.println(names[1]); // Bob
```

7 Multidimensional Array (Example)

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(matrix[1][2]); // 6
```

8 Key Takeaways

- An array stores multiple values of the same type.
- Array elements have indexes starting from 0.
- The array can be created with a specified size or initialized immediately with values.
- The length of an array is obtained using **length**.
- Arrays are often used for processing collections of data (searching, sorting, calculating sums, finding minimum/maximum values, etc.).

9 Try It Yourself



1. Given an array {8, 15, 3, 22, 7}, find and output the minimum value.
2. Given an array of 5 numbers, calculate their sum and average value.
3. Given an array of integers, count how many even and odd numbers it contains.
4. Given an array, output all its elements in reverse order.

Working with Arrays in Java

Indices, length, and traversing arrays



An array is a collection of elements of the same type, which can be accessed by index. Indices start at 0.



1 Basic Concepts

- An array stores elements of the same type (e.g., int, String).
- Indices start at 0.
- The **length** of an array is given by the length property.
- Elements can be accessed using indices with loops (for, enhanced for).

Indices:	0	1	2	3	4
Elements:	10	20	30	40	50

For an array of 5 elements, the last index is 4 (length - 1).

2 Ways to Traverse an Array

1 Classic for loop (by index)

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

2 Enhanced for loop (for-each)

```
for (int x : arr) {
    System.out.println(x);
}
```



Important Notes

- Use arr.length, not a fixed number of elements.
- Indices range from 0 to arr.length - 1.
- Check array bounds to avoid ArrayIndexOutOfBoundsException.

3 Classic Task

Task: Find the maximum element in an array of integers.

Given an array of integers, you need to:

1. Output all elements of the array.
2. Find and output the maximum element.
3. Calculate and output the sum of the elements.
4. Calculate and output the average value.

4 Java Solution Example

```
1 public class ArrayExample {
2     public static void main(String[] args) {
3         int[] numbers = {12, 5, 18, 7, 25, 10};
4
5         // 1. Print all elements
6         System.out.println("Array elements:");
7         for (int i = 0; i < numbers.length; i++) {
8             System.out.print(numbers[i] + " ");
9         }
10        System.out.println();
11
12        // 2. Find the maximum and sum
13        int max = numbers[0];
14        int sum = 0;
15        for (int i = 0; i < numbers.length; i++) {
16            if (numbers[i] > max) {
17                max = numbers[i];
18            }
19            sum += numbers[i];
20        }
21
22        // 3. Calculate the average
23        double average = (double) sum / numbers.length;
24
25        // 4. Output the results
26        System.out.println("Maximum element: " + max);
27        System.out.println("Sum of elements: " + sum);
28        System.out.println("Average value: " + average);
29    }
30 }
```

5 Program Output Example

Array elements:

12 5 18 7 25 10

Maximum element: 25

Sum of elements: 97

Average value: 16.166666666666668

6 Other Useful Tasks with Arrays

- 1 Find the minimum element of an array.
- 2 Count how many elements are greater than a given number.
- 3 Find the index of a given element (if it exists).
- 4 Reverse the array (in place).
- 5 Create a new array containing only even (or only positive) elements.

7 Additional Example: User Input

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter the array size: ");
int n = sc.nextInt();
int[] arr = new int[n];

System.out.println("Enter the array elements:");
for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}
```



Discussion Questions

- What happens if you access arr[arr.length]?
- How can you find two maximum elements in an array?
- How can you create an array of random numbers?

During the lecture, complete the task: “Analysis of an Integer Array.” You have 10–15 minutes.

During the lecture, complete the task: “Analysis of an Integer Array.” You have 10–15 minutes.

First, understand the task. What does this program calculate?

Then complete the task step by step.

Please use the given source code. This code has already been checked and tested.

You do not need to use AI for this task. There is no need for it. The program uses the operators and commands that we have studied in this lecture.

Try to understand each line of the program and how it works. This is how you learn programming.

I learned programming in the same way — by working with ready-made examples, running them, understanding them, and then changing them.

New Project

New Project

- Java
- Kotlin
- Groovy
- Empty Project

Generators

- Maven Archetype
- Jakarta EE
- Spring Boot
- JavaFX
- Quarkus
- Micronaut
- Ktor
- Compose for Desktop
- HTML
- React
- Express
- Angular CLI
- Vue.js
- Vite

More via plugins...



Name: sept2w_1

Location: ~\IdeaProjects

Project will be created in: ~\IdeaProjects\sept2w_1

☐ Create Git repository

Build system: IntelliJ Maven Gradle

JDK: 21 Oracle OpenJDK 21.0.6

☒ Add sample code

☐ Generate code with onboarding tips

> Advanced Settings

Create

Cancel

```

import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, -5, 0, 8, -2, 15, 3};
        int max = numbers[0];
        int min = numbers[0];
        int sum = 0;
        int positive = 0;
        int negative = 0;
        int zero = 0;

        // process the array
        for (int num : numbers) {

            if (num > max) max = num;
            if (num < min) min = num;

            sum += num;

            if (num > 0) {
                positive++;
            } else if (num < 0) {
                negative++;
            } else {
                zero++;
            }
        }
        double average = (double) sum / numbers.length;
        // output results
        System.out.println("Array: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
        System.out.println("Maximum value : " + max);
        System.out.println("Minimum value : " + min);
        System.out.println("Sum : " + sum);
        System.out.println("Average value : " + average);
        System.out.println("Positive numbers: " + positive);
        System.out.println("Negative numbers: " + negative);
        System.out.println("Zero numbers : " + zero);
    }
}

```

```

1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         int[] numbers = {10, -5, 0, 8, -2, 15, 3};
5         int max = numbers[0];
6         int min = numbers[0];
7         int sum = 0;
8         int positive = 0;
9         int negative = 0;
10        int zero = 0;
11
12        // process the array
13        for (int num : numbers) {
14
15            if (num > max) max = num;
16            if (num < min) min = num;
17
18            sum += num;
19
20            if (num > 0) {
21                positive++;
22            } else if (num < 0) {
23                negative++;

```

S1 sept2w_1 Version control

Main

Project

sept2w_1 C:\Users\WW\IdeaProjects\sept2w_1

.idea

out

src

Main

.gitignore

sept2w_1.iml

External Libraries

Main.java

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         int[] numbers = {10, -5, 0, 8, -2, 15, 3};
5         int max = numbers[0];
6         int min = numbers[0];
7         int sum = 0;
8         int positive = 0;
9         int negative = 0;
```

Run Main

Run

Debug

Profile

Test

Run with Parameters

Run with Coverage

Run with Profiler

Run with JUnit

Run with JUnit 5

Run with JUnit 4

Run with JUnit 3

Run with JUnit 2

Run with JUnit 1

Run with JUnit 0

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ Idea2024

Array:

10 -5 0 8 -2 15 3

Maximum value : 15

Minimum value : -5

Sum : 29

Average value : 4.142857142857143

Positive numbers: 4

Negative numbers: 2

Zero numbers : 1

Process finished with exit code 0

Performance

Show Results

4

Practical Task (Classwork)

Array of integers: find values and calculate statistics



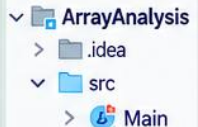
Analyze an array of integers.
Find the maximum, minimum,
average value and count positive,
negative and zero elements.

1 Start IntelliJ IDEA



- 1 Open IntelliJ IDEA.
- 2 On the Welcome screen click **New Project**.
- 3 Select **Java** and click **Next**.
- 4 Give the project name: **ArrayAnalysis**.
- 5 Choose a location and click **Create**.

2 Create a Java Class



- 1 Right-click the **src** folder.
- 2 Select **New** → **Java Class**.
- 3 Enter the class name: **Main**.
- 4 Click **Enter**.

3 Write the Code

Copy and paste the code on the right into **Main.java**.

4 Run the Program



- 1 Click the green **Run** button (▶).
- 2 See the output in the **Run** window.

5 Complete Java Code (Main.java)

```

1  import java.util.Scanner;
2
3  public class Main {
4      public static void main(String[] args) {
5          int[] numbers = {10, -5, 0, 8, -2, 15, 3};
6
7          int max = numbers[0];
8          int min = numbers[0];
9          int sum = 0;
10         int positive = 0;
11         int negative = 0;
12         int zero = 0;
13
14         // process the array
15         for (int num : numbers) {
16             if (num > max) max = num;
17             if (num < min) min = num;
18             sum += num;
19
20             if (num > 0) {
21                 positive++;
22             } else if (num < 0) {
23                 negative++;
24             } else {
25                 zero++;
26             }
27         }
28
29         double average = (double) sum / numbers.length;
30
31         // output results
32         System.out.println("Array: ");
33         for (int num : numbers) {
34             System.out.print(num + " ");
35         }
36         System.out.println();
37         System.out.println("Maximum value : " + max);
38         System.out.println("Minimum value : " + min);
39         System.out.println("Sum : " + sum);
40         System.out.println("Average value : " + average);
41         System.out.println("Positive numbers: " + positive);
42         System.out.println("Negative numbers: " + negative);
43         System.out.println("Zero numbers : " + zero);
44     }
45 }
    
```

6 Example Output

```

Array:
10 -5 0 8 -2 15 3

Maximum value : 15
Minimum value : -5
Sum : 29
Average value : 4.142857142857143
Positive numbers: 4
Negative numbers: 2
Zero numbers : 1
    
```

7 Task for Students

- 1 Run the given code and make sure you get the same result.
- 2 Change the array values and run the program again.
- 3 Modify the program to enter the numbers from the keyboard instead of using a fixed array.
- 4 Add a new requirement: count how many numbers are greater than a given value (e.g. > 10).
- 5 Add a menu that allows the user to choose:
 - find maximum
 - find minimum
 - find average
 - count positive/negative/zero numbers
 - exit

8 Extra (Optional)

- 1 Find and print all even numbers from the array.
- 2 Find and print the sum of only positive numbers.

Homework / SIWT Assignment/ Office Hours Assignment

Complete ONE of the six variants.

Practical Assignment. Three Variants (Desktop Applications)

Java Swing – Three Variants



Goal:

Create a desktop application with a convenient interface to study the covered topics.

1 Variant 1. Student Manager

Create a desktop application to manage a list of students. The data is stored in memory during the program execution.

No	Name	Score
1	Amina	85
2	Bob	92
3	Tom	76

Students: 3 Average score: 84.3

Main Requirements:

1. Use an array (name, score).
2. Menu (buttons) for operations.
3. Add, view and search by name.
4. Calculate the average score.
5. Find the student with the highest score.
6. Delete a student.
7. Use **if**, **switch** for menu/cases.
8. Nice interface (JFrame, JButton, JTable, JTextField).

2 Variant 2. Electronics Store

Create a desktop application "Electronics Store". The user can view the product list, specify quantity, add items to the shopping cart and complete the purchase. Product data is stored in memory.

No	Name	Price	In stock
1	Laptop	450000	5
2	Mouse	7000	10
3	Keyboard	12000	8
4	Monitor	90000	4
5	Headphones	25000	6

Product	Qty	Sum
Mouse	2	14000
Keyboard	1	12000

Quantity: 1 Total: 26000

Buttons: Add to Cart, Buy, Clear Cart

Main Requirements:

1. Product array (name, price, quantity).
2. View item details.
3. Add item to cart (array or list).
4. Check total purchase amount.
5. Display cart contents.
6. Option to buy items and clear the cart.
7. Use **if-else**, **switch** for actions.
8. Nice interface (JPanel, JTable, JButton, JLabel, JTextField, JComboBox, JSpinner).

3 Variant 3. Array Calculator

Create a desktop application to work with an array of numbers. The user can enter the array size and elements, and then perform different operations.

Input data: Array size: 5 Create array

Elements (space separated): 10 3 7 8 1 Load

Array: [10, 3, 7, 8, 1]

Result: Sum: 29, Average: 5.8, Minimum: 1, Maximum: 10, Even: 2, Odd: 3, Sorted: [1, 3, 7, 8, 10]

Operations: Sum, Average, Minimum, Maximum, Count even, Sort

Find number: 7

Main Requirements:

1. Enter array size and its elements.
2. Calculate sum, average value.
3. Find minimum and maximum.
4. Count even and odd numbers.
5. Sort array (e.g. by swapping).
6. Search for a given number (present or not).
7. Use arrays, loops, operators, **if**, **break**, **continue**, sorting (e.g. bubble sort).
8. Nice interface (JFrame, JButton, JTextField, JTextArea).



Topics to Use (as many as possible)

- Arrays: creation, initialization, traversal
- Conditional statements: **if**, **else if**, **switch**
- Loops: **for**, **while**, **do-while**
- Operators: arithmetic, comparison, logical
- Methods (with parameters and return values)
- Working with strings (String)
- Input (JTextField, JOptionPane)
- Swing components (JFrame, JPanel, JButton, JTable, JLabel, etc.)



Additional Ideas (optional)

- Save/load data to a file (txt).
- Use classes (Student, Product).
- Check incorrect input and show error message.
- Improve the interface (colors, fonts, icons).



Result

A working desktop application in Java implementing the chosen variant.

Practical Assignment. More Variants

Desktop Applications (Java Swing)



Choose **ONE** of the variants (4, 5 or 6) and develop a desktop application. Use as many of the studied topics as possible: arrays, conditions, loops, operators, methods.

4 Variant 4. Unit Converter

Create a desktop application for converting different units of measurement.

The screenshot shows a Java Swing window titled "Unit Converter". It has a category dropdown set to "Length", a "From" dropdown set to "Kilometer (km)", and a "To" dropdown set to "Meter (m)". A text field for "Value" contains "2.5". A "Convert" button is below the value field. To the right, under "Popular units:", there is a list of units: Length: mm, cm, m, km; Mass: g, kg, ton; Time: sec, min, hour; Temperature: °C, °F, K; Area: m², km², ha; Volume: ml, l, m³. At the bottom, a "Result" field shows "2500.0 m" and a "Clear" button.

Main Functions:

1. Choose a category (length, mass, time, temperature, area, volume).
2. Select units "from" and "to".
3. Enter a value and get the result.
4. Check input correctness (number, positive value).
5. Possibility to add new units (stored in an array).
6. Use **switch** for choosing a formula or **if - else**.
7. Buttons: Convert, Clear, Exit.



Topics to Use (as many as possible)

- Arrays (creation, initialization, traversal)
- Loops: **for**, **while**, **do-while**
- Conditional statements: **if**, **else if**, **switch**
- Operators: arithmetic, comparison, logical
- Methods (with parameters and return values)
- Working with strings (String)
- Swing components (JFrame, JPanel, JButton, JTable, JLabel, JTextField, JComboBox, etc.)
- Event handling
- Input validation
- Nice and clear user interface

5 Variant 5. Student Directory

Create a desktop application to work with student data.

The screenshot shows a Java Swing window titled "Student Directory". It has a "Student Data" section with fields for Name (Amina), Group (B057-01), and Score (85). There are "Add", "Update", "Delete", and "Clear" buttons. To the right is a "Search" section with a text field containing "Amina" and a "Search" button. Below that is a "Filter" section with a dropdown menu set to "All groups". At the bottom is a "Student List" table with columns No, Name, Group, and Score. The table contains 4 rows of data. Below the table, it says "Total students: 4" and "Average score: 85.3". There is an "Exit" button at the bottom right.

No	Name	Group	Score
1	Amina	B057-01	85
2	Bob	B057-01	92
3	Tom	B057-02	76
4	Sara	B057-02	88

Main Functions:

1. Add, edit and delete student data.
2. View all data in a table.
3. Search a student by name (use a loop).
4. Filter students by group (use conditional operators).
5. Calculate the average score and find the student with the highest score.
6. Sort the list by score (optional).
7. Nice interface: table (JTable), buttons, input fields.



Additional Ideas (optional)

- Save/load data to a file (txt).
- Use classes (e.g., Student, Product).
- Add images (icons).
- Show an error message for incorrect input.
- Add help information (short instructions).
- Extend the functionality with your own ideas.

6 Variant 6. Calculator with History

Create a desktop application with basic mathematical operations and a calculation history.

The screenshot shows a Java Swing window titled "Smart Calculator". It has a large display area showing "125 * 3" and the result "375". Below the display is a numeric keypad with buttons for digits 0-9, a decimal point, and basic operators (+, -, *, /). There are also buttons for "C (Clear)", "Delete", and "Clear History". To the right is a "Calculation History" panel with a list of previous calculations: 12 + 8 = 20, 50 / 5 = 10, 7 * 6 = 42, 100 - 25 = 75, 25 * 4 = 100, and 125 * 3 = 375. A scroll bar is on the right of the history panel.

Main Functions:

1. Perform operations: +, -, *, /.
2. Enter numbers using buttons (like a regular calculator).
3. Use an array or a string list to store the calculation history.
4. Check division by zero.
5. Possibility to clear current input and the whole history.
6. Use **if - else**, **switch** (if needed).
7. Nice interface: buttons, text fields, history area.



Result

A working desktop application in Java (Swing) implementing one of variants (4, 5 or 6) and demonstrating your skills.

Stratospheric Skydive: Sergey Boytsov



Stratospheric Jump

Sergey Boytsov made a stratospheric jump from a helium balloon aircraft at a height of **11.5 kilometers**.



Free Fall

The free fall lasted for about **2.5 minutes**, and the speed reached **501 km/h**.



Temperature

At this altitude, the temperature dropped to about **-50...-60 °C**.



Parachute and Landing

The athlete opened the parachute at a height of **1.5 kilometers** above the ground. The landing was successful.